

4

Lenguajes Formales y Automatas

Clave: 0442

3

Gramáticas regulares y autómatas de estado finito

- Objetivo: El alumno explicará los conceptos de autómatas finitos y gramáticas regulares a través de la relación entre los autómatas finitos, los no determinísticos y las gramáticas regulares.

Contenido:

- 3.1 Introducción a las gramáticas regulares
- 3.2 Autómata finito determinístico
- 3.3 Autómata finito no-determinístico
- 3.4 Autómata finito con movimientos ϵ
- 3.5 Equivalencia y minimización de autómatas finitos
- 3.6 Gramática regular a partir de un autómata finito

3.1

Introducción a las gramáticas regulares

- Una gramática es un conjunto de reglas.
- Una regla es una expresión de la forma $\alpha \rightarrow \beta$, en donde tanto α como β son cadenas de símbolos en donde pueden aparecer tanto elementos del alfabeto Σ como unos nuevos símbolos, llamados variables. Los símbolos que no son variables son constantes.
- En compiladores, a las variables se les conoce como símbolos No Terminales y a las constantes como símbolos Terminales.
- Nos vamos a interesar por el momento en las gramáticas cuyas reglas son de la forma $A \rightarrow aB$ o bien $A \rightarrow a$, donde A y B son variables, y a es un caracter terminal. A estas gramáticas se les llama regulares.

3.1

Introducción a las gramáticas regulares

- Sea una gramática con las siguientes reglas:
- 1. $S \rightarrow aA$
- 2. $S \rightarrow bA$
- 3. $A \rightarrow aB$
- 4. $A \rightarrow bB$
- 5. $A \rightarrow a$
- 6. $B \rightarrow aA$
- 7. $B \rightarrow bA$



3.1

Introducción a las gramáticas regulares

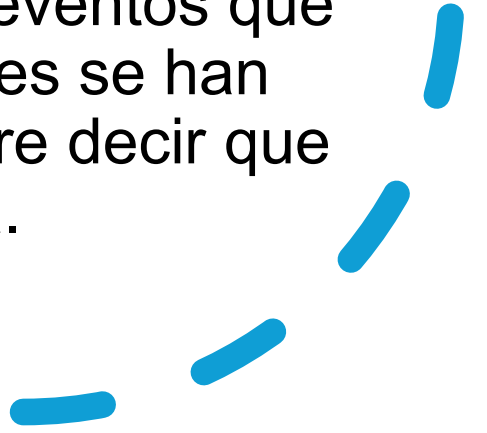
- La idea para aplicar una gramática es que se parte de una variable, llamada símbolo inicial, y se aplican repetidamente las reglas gramaticales, hasta que ya no haya variables en la palabra. En ese momento se dice que la palabra resultante es generada por la gramática, o en forma equivalente, que la palabra resultante es parte del lenguaje de esa gramática.
- Por ejemplo, en la gramática que acabamos de presentar, si consideramos que las variables son S (que será el símbolo inicial), A y B , y las constantes a y b , partiendo de S Podemos producir bA (por la segunda regla), luego de bA podemos pasar a ba (por la quinta regla). Como ba tiene sólo constantes, podemos concluir que la palabra ba es parte del lenguaje generado por la gramática dada. De hecho el lenguaje generado por esta gramática es el de las palabras en $\{a, b\}$ de longitud par terminadas en a .

3.1

Introducción a las gramáticas regulares

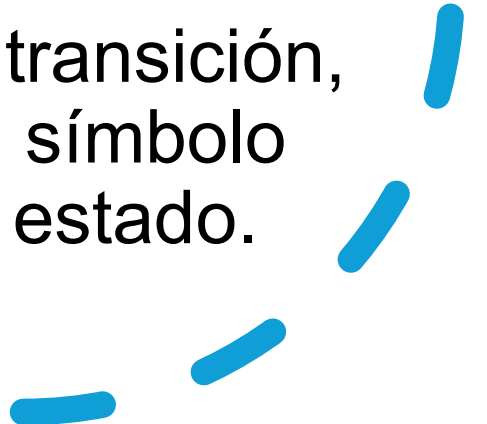
- Definición.- Una gramática regular es un cuádruplo (V, Σ, R, S) en donde:
- V es un alfabeto de variables,
- Σ es un alfabeto de constantes,
- R , el conjunto de reglas, es un subconjunto finito de $V \times (\Sigma V \cup \Sigma)$.
- S , el símbolo inicial, es un elemento de V .
- Por ejemplo, la gramática que presentamos arriba se representaría formalmente como:
- $(\{S, A, B\}, \{a, b\}, \{(S, aA), (S, bA), (A, aB), (A, bB), (A, a), (B, aA), (B, bA)\}, S)$

3.2 Autómata finito determinístico

- El funcionamiento de los autómatas finitos consiste en ir pasando de un estado a otro, a medida que va recibiendo los caracteres de la palabra de entrada. Este proceso puede ser seguido fácilmente en los diagramas de estados. Simplemente hay que pasar de estado a estado siguiendo las flechas de las transiciones, para cada caracter de la palabra de entrada, empezando por el estado inicial.
 - Los estados son el único medio de que disponen los AF para recordar los eventos que ocurren (por ejemplo, qué caracteres se han leído hasta el momento); esto quiere decir que son máquinas de memoria limitada.
- 

3.2 Autómata finito determinístico

Definición.- Una máquina de estados finitos M es un quintuplo $(K, \Sigma, \delta, s, F)$, donde:

- K es un conjunto de identificadores (símbolos) de estados;
 - Σ es el alfabeto de entrada;
 - $s \in K$ es el estado inicial;
 - $F \subseteq K$ es un conjunto de estados finales;
 - $\delta : K \times \Sigma \rightarrow K$ es la función de transición, que a partir de un estado y un símbolo del alfabeto obtiene un nuevo estado.
- 

3.2 Autómata finito determinístico

- Ejemplo.- Un autómata finito determinista puede ser expresado formalmente como: $M = (K, \Sigma, \delta, q_0, F)$, donde:
- $K = \{q_0, q_1, q_2\}$
- $\Sigma = \{a, b\}$
- $\delta = \{((q_0, a), q_1), ((q_0, b), q_2), ((q_1, a), q_1), ((q_1, b), q_1), ((q_2, a), q_0), ((q_2, b), q_2)\}$
- $F = \{q_1, q_2\}$



3.3 Autómata finito no- determinístico

- Una extensión a los autómatas finitos deterministas es la de permitir que de cada nodo del diagrama de estados salga un número de flechas mayor o menor que $|\Sigma|$. Así, se puede permitir que falte la flecha correspondiente a alguno de los símbolos del alfabeto, o bien que haya varias flechas que salgan de un sólo nodo con la misma etiqueta. Inclusive se permite que las transiciones tengan como etiqueta palabras de varias letras o hasta la palabra vacía. A estos autómatas finitos se les llama no determinísticos o no deterministas (abreviado AFN).

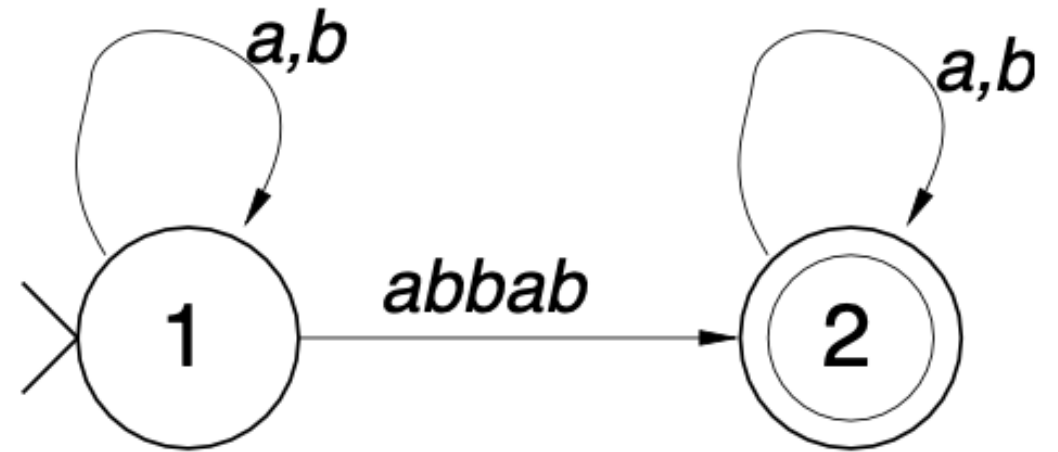
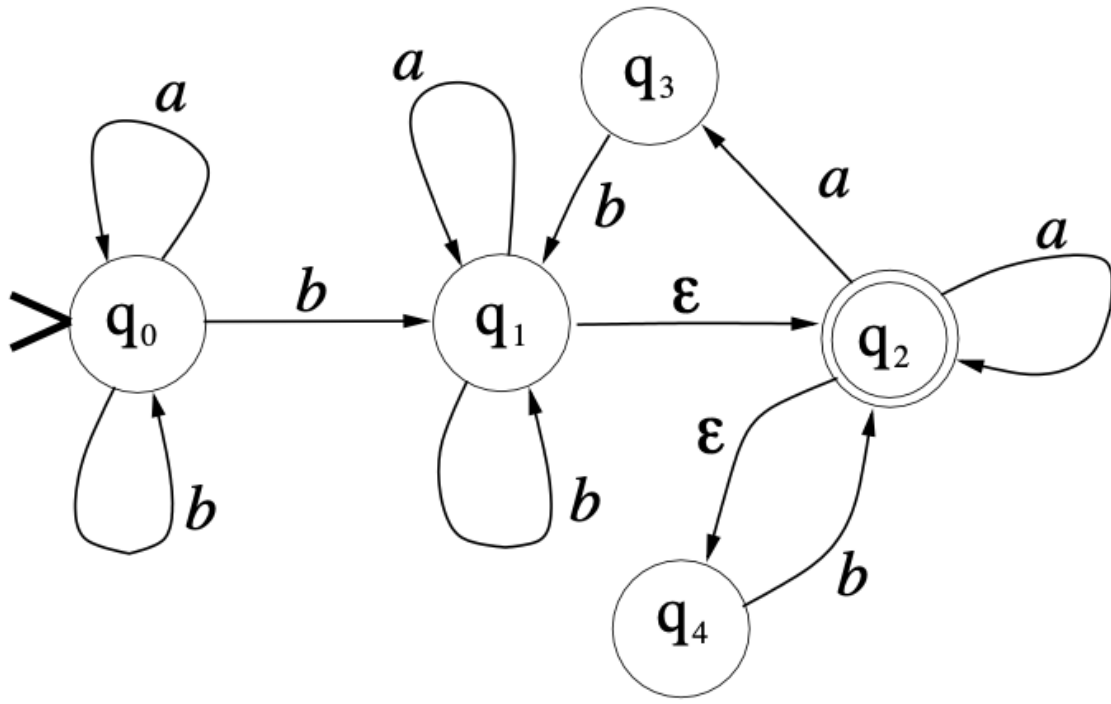
3.3 Autómata finito no- determinístico

Definición.- Un autómata finito no determinista es un quintuplo $(K, \Sigma, \Delta, s, F)$ donde K , Σ , s y F tienen el mismo significado que para el caso de los autómatas determinísticos, y Δ , llamado la relación de transición, es un subconjunto finito de $K \times \Sigma^* \times K$.

Por ejemplo, un AFN quedaría representado matemáticamente por el siguiente quintuplo:

- $(\{1, 2\}, \{a, b\}, \{(1, a, 1), (1, b, 1), (1, abbab, 2), (2, a, 2), (2, b, 2)\}, 1, \{2\})$
- El punto esencial es que Δ es una relación, no una función.

3.3 Autómata finito no-determinístico



A close-up photograph of a snake's head, showing its large, dark eye and the intricate pattern of its orange and white scales. The snake is coiled, with its body visible in the foreground. The background is dark and out of focus.

Programa python

```
class DFA:
    def __init__(self, Q, sigma, delta, q0, F):
        self.Q = Q # set of states
        self.sigma = sigma # set of symbols
        self.delta = delta # transition function as a dictionary
        self.q0 = q0 # initial state
        self.F = F # set of final states

    def __repr__(self):
        return f"DFA({self.Q},\n\t{self.sigma},\n\t{self.delta},\n\t{self.q0})"

    def run (self, w):
        q = self.q0
        while w != "":
            print(w[0])
            q = self.delta[(q, w[0])]
            w = w[1:]
        return q in self.F
```

```
if __name__ == '__main__':
```

```
    D0 = DFA({0,1,2},  
             {"a","b"},  
             {(0, "a"): 0, (0, "b"): 1,  
              (1, "a"): 2, (1, "b"): 1,  
              (2, "a"): 2, (2, "b"): 2},  
             0,  
             {0,1})
```

```
word = "aabb"  
print(D0)  
print(word)  
print(D0.run(word))
```



Programa java

```
public class Tupla
{
    private Integer state;
    private String character;
    private int hashCode;

    public Tupla(Integer estado, String character)
    {
        this.state = estado;
        this.character = character;
        this.hashCode = Objects.hash(estado, character);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (o == null || getClass() != o.getClass())
            return false;
        Tupla that = (Tupla) o;
        return state.equals(that.state) && character.equals(that.character);
    }

    @Override
    public int hashCode() {
        return this.hashCode;
    }
}
```

```
public class DFA
{
    int q0;
    Set<Integer> F;
    HashMap<Tupla, Integer> delta;

    public DFA(int q0, Set<Integer> F, HashMap<Tupla, Integer> delta)
    {
        this.q0 = q0;
        this.F = F;
        this.delta = delta;
    }

    public boolean run(String w)
    {
        int q = this.q0;
        while(!w.isBlank())
        {
            Tupla temp = new Tupla(q, w.substring(0,1));
            q = this.delta.get(temp);
            w = w.substring(1);
        }
        return F.contains(q);
    }
}
```

```
public class Main {  
    public static void main(String[] args)  
    {  
        Set<Integer> F = new TreeSet<>();  
        F.add(0);  
        F.add(1);  
  
        HashMap<Tupla, Integer> delta = new HashMap<>();  
        delta.put(new Tupla(0,"a"), 0);  
        delta.put(new Tupla(0,"b"), 1);  
        delta.put(new Tupla(1,"a"), 2);  
        delta.put(new Tupla(1,"b"), 1);  
        delta.put(new Tupla(2,"a"), 2);  
        delta.put(new Tupla(2,"b"), 2);  
  
        DFA D0 = new DFA(0,F,delta);  
        System.out.println(D0.run("aaabb"));  
    }  
}
```

Programa 1

- Palabras que empiecen por "a" y terminen con "c" con alfabeto = {a,b,c,d}
- Palabras que contengan "ala" con alfabeto = {a,e,i,o,u,l,s}.

Fin